

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN
A DISTANCIA – UTN

Trabajo Práctico Integrador

Gestión de Datos de Países en Python:
Filtros, Ordenamientos y Estadísticas

Materia: Programación 1

Carrera: Tecnicatura Universitaria en Programación

Institución: Universidad Tecnológica Nacional (UTN)

Integrantes del equipo:

Estudiante 1 – [Nombre completo]

Estudiante 2 – [Nombre completo]

Fecha de entrega: Junio 2026

Índice

1. Introducción	3
2. Marco Teórico	3
2.1 Listas	3
2.2 Diccionarios	4
2.3 Funciones	4
2.4 Estructuras Condicionales y Repetitivas	4
2.5 Ordenamientos	5
2.6 Estadísticas Básicas	5
2.7 Archivos CSV	5
3. Decisiones Técnicas y Arquitectura	6
3.1 Estructura del Proyecto	6
3.2 Flujo de Operaciones Principales	6
3.3 Estructura de Datos Elegida	7
4. Funcionalidades del Sistema	7
5. Fragmentos de Código Relevantes	8
6. Dificultades y Conclusiones	10
7. Bibliografía / Webgrafía	10

1. Introducción

El presente Trabajo Práctico Integrador tiene como objetivo desarrollar una aplicación en Python que permita gestionar información sobre países del mundo, aplicando los conceptos fundamentales de Programación 1: listas, diccionarios, funciones, estructuras condicionales y repetitivas, algoritmos de ordenamiento y estadísticas básicas.

El sistema permite leer y escribir datos desde un archivo CSV, realizar búsquedas, filtrados, ordenamientos y mostrar estadísticas a través de un menú interactivo en consola. El diseño del código prioriza la modularización: cada función cumple una única responsabilidad, facilitando la legibilidad y el mantenimiento.

2. Marco Teórico

2.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar colecciones de elementos de cualquier tipo. Se definen con corchetes: **lista = []**. En este proyecto, la lista principal (*lista_paises*) almacena todos los países cargados desde el CSV, donde cada elemento es un diccionario.

Las listas permiten operaciones como **append()** para agregar elementos, iteración con **for**, acceso por índice y ordenamiento con algoritmos manuales. Son la estructura base sobre la que se aplican todos los filtros y ordenamientos del sistema.

2.2 Diccionarios

Un diccionario es una colección de pares clave-valor, no ordenada (en versiones antiguas de Python) y mutable. Se define con llaves: **{'clave': valor}**. En este proyecto, cada país se representa como un diccionario con las claves: *nombre*, *poblacion*, *superficie* y *continente*.

El uso de diccionarios facilita el acceso semántico a los datos (**pais['nombre']** en lugar de **pais[0]**), mejorando la legibilidad del código y reduciendo la probabilidad de errores por confusión de índices.

2.3 Funciones

Las funciones son bloques de código reutilizables que encapsulan una tarea específica. Se definen con la palabra clave **def**. El principio de modularización establece que cada función debe tener una única responsabilidad, lo que facilita el testing, el mantenimiento y la comprensión del código.

En este proyecto se implementaron funciones específicas para cada operación del sistema: lectura del CSV, escritura, agregar, buscar, actualizar, filtrar, ordenar y mostrar estadísticas. Esto permite modificar cualquier funcionalidad sin afectar el resto.

2.4 Estructuras Condicionales y Repetitivas

Las estructuras condicionales (**if / elif / else**) permiten ejecutar bloques de código según condiciones lógicas. Las estructuras repetitivas (**for** y **while**) permiten iterar sobre colecciones o repetir acciones hasta que se cumpla una condición.

En el sistema se utilizan bucles **for** para recorrer la lista de países, y bucles **while True** combinados con validaciones para garantizar que el usuario ingrese datos correctos antes de continuar. Los condicionales permiten ramificar la lógica según el criterio de filtrado u ordenamiento seleccionado.

2.5 Ordenamientos

El algoritmo de ordenamiento de burbuja (Bubble Sort) compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Este proceso se repite hasta que la lista queda completamente ordenada. Su complejidad temporal es $O(n^2)$.

Se eligió Bubble Sort por su simplicidad de implementación y comprensión, adecuada para el nivel de la materia. El sistema permite ordenar por nombre (string), población o superficie (enteros), tanto de forma ascendente como descendente.

2.6 Estadísticas Básicas

Las estadísticas descriptivas permiten resumir y caracterizar un conjunto de datos. En este proyecto se calculan: el país con mayor y menor población, el promedio de población y de superficie, y la cantidad de países por continente. Estos indicadores se obtienen mediante una única iteración sobre la lista, acumulando valores.

2.7 Archivos CSV

Un archivo CSV (Comma-Separated Values) es un formato de texto plano donde cada línea representa un registro y los campos se separan por comas. Python permite trabajar con ellos mediante las funciones nativas **open()**, **readline()**, **readlines()** y métodos de string como **split(',')** y **strip()**.

En este proyecto se implementó la lectura y escritura manual del CSV sin usar la biblioteca csv de Python, aplicando directamente operaciones de string para parsear cada línea y convertir los tipos de datos correspondientes.

3. Decisiones Técnicas y Arquitectura

3.1 Estructura del Proyecto

Archivo	Descripción
países.csv	Dataset base con los países del sistema
main.py	Código fuente principal con todas las funciones y el menú
README.md	Documentación del repositorio con instrucciones de uso

3.2 Flujo de Operaciones Principales

El flujo general del sistema sigue los siguientes pasos al iniciarse:

1. Se carga el archivo CSV en memoria como una lista de diccionarios (*enlistar_csv*).
2. Se muestra el menú principal con las opciones disponibles.
3. El usuario selecciona una opción; el sistema valida la entrada.
4. Se ejecuta la función correspondiente, operando sobre la lista en memoria.
5. Las operaciones que modifican datos (agregar, actualizar) guardan los cambios en el CSV (*guardar_csv*).
6. El menú se repite hasta que el usuario elige salir (opción 0).

3.3 Estructura de Datos Elegida

Se eligió una **lista de diccionarios** como estructura principal. Cada diccionario representa un país con sus cuatro atributos. Esta decisión permite:

- Acceso semántico a los datos: **pais['nombre']** es más claro que **pais[0]**.
- Fácil extensibilidad: agregar un nuevo campo solo requiere una nueva clave.
- Compatibilidad directa con el formato CSV mediante conversión de tipos.
- Iteración simple y legible con bucles **for**.

Ejemplo de un país en memoria:

```
{
    'nombre'      : 'Argentina',
    'poblacion'   : 45376763,
    'superficie'  : 2780400,
    'continente'  : 'America'
}
```

4. Funcionalidades del Sistema

Función	Descripción	Validaciones aplicadas
enlistar_csv()	Lee el CSV y devuelve lista de diccionarios	FileNotFoundError, formato incorrecto por línea
guardar_csv()	Sobreescribe el CSV con la lista actualizada	—
agregar_pais()	Solicita y valida datos de un nuevo país	Campos vacíos, nombre duplicado, tipos numéricos, valores positivos
buscar_pais()	Búsqueda parcial o exacta por nombre	Término vacío, sin resultados
actualizar_datos()	Actualiza población o superficie de un país	País no encontrado, valores inválidos o negativos
filtrar_paises()	Filtra por continente, rango de población o superficie	Opción inválida, sin resultados
ordenar_paises()	Ordena con Bubble Sort por nombre, población o superficie	Criterio inválido, dirección inválida
mostrar_estadisticas()	Muestra indicadores clave del dataset	Lista vacía
mostrar_pais()	Helper para mostrar un país con formato	—
menu()	Punto de entrada, bucle principal del programa	Opción inválida, ValueError en entrada

5. Fragmentos de Código Relevantes

5.1 Lectura del CSV con manejo de errores de formato

```
def enlistar_csv(nombre_archivo):
    lista = []
    try:
        archivo = open(nombre_archivo, 'r', encoding='utf-8')
    except FileNotFoundError:
        print(f'Error: No se encontro el archivo "{nombre_archivo}"')
        return lista

    archivo.readline() # Saltar encabezado
    for linea in archivo:
        linea_limpia = linea.strip()
        if not linea_limpia:
            continue
        try:
            nombre, poblacion, superficie, continente = linea_limpia.split(',')
            pais = {
                'nombre' : nombre,
                'poblacion' : int(poblacion),
                'superficie' : int(superficie),
                'continente' : continente,
            }
            lista.append(pais)
        except ValueError:
            print(f'Advertencia: linea con formato incorrecto -> {linea_limpia}')
    archivo.close()
    return lista
```

5.2 Validación de entrada con bucle while

```
# Ejemplo: validacion de poblacion en agregar_pais()
while True:
    entrada = input('Poblacion: ').strip()
    if entrada == '':
        print('Error: la poblacion no puede estar vacia.')
        continue
    try:
        poblacion = int(entrada)
        if poblacion <= 0:
            print('Error: la poblacion debe ser mayor a 0.')
        else:
            break
    except ValueError:
        print('Error: ingrese un numero entero valido.')
```

5.3 Algoritmo Bubble Sort con criterio y dirección variables

```
def ordenar_paises(lista_paises):
    # ...validaciones de criterio y direccion...
    claves = {1: 'nombre', 2: 'poblacion', 3: 'superficie'}
    clave = claves[criterio]
    n = len(lista_paises)
```

```

for i in range(n):
    for j in range(n - 1 - i):
        a = lista_paises[j][clave]
        b = lista_paises[j + 1][clave]
        if isinstance(a, str):
            a, b = a.lower(), b.lower()
        intercambiar = (a > b) if direccion == 'asc' else (a < b)
        if intercambiar:
            lista_paises[j], lista_paises[j+1] = lista_paises[j+1], lista_paises[j]

```

5.4 Cálculo de estadísticas en una sola pasada

```

mayor_poblacion = lista_paises[0]['poblacion']
menor_poblacion = lista_paises[0]['poblacion']
suma_poblacion = 0
suma_superficie = 0
conteo = {}

for pais in lista_paises:
    pob = pais['poblacion']
    con = pais['continente']
    if pob > mayor_poblacion:
        mayor_poblacion = pob
    if pob < menor_poblacion:
        menor_poblacion = pob
    suma_poblacion += pob
    suma_superficie += pais['superficie']
    conteo[con] = conteo.get(con, 0) + 1

promedio_poblacion = suma_poblacion / len(lista_paises)
promedio_superficie = suma_superficie / len(lista_paises)

```


6. Dificultades y Conclusiones

Dificultades encontradas

- Manejo de la indentación en Python: al ser un lenguaje donde la indentación es parte de la sintaxis, fue necesario prestar especial atención al anidar bucles, condicionales y bloques try/except correctamente.
- Validación de tipos de datos: garantizar que el usuario no ingrese letras donde se esperan números (y viceversa) requirió combinar try/except con bucles while, lo que implicó comprender bien el flujo de control del programa.
- Algoritmo Bubble Sort con múltiples criterios: adaptar el algoritmo para que compare por diferentes campos (string vs entero) y en ambas direcciones requirió un diseño cuidadoso del bloque de comparación.
- Manejo del archivo CSV: distinguir cuándo releer el archivo vs. operar sobre la lista en memoria, y garantizar que los cambios se persistan correctamente con `guardar_csv()`, fue un aspecto que requirió planificación previa.

Conclusiones

Este proyecto permitió integrar de forma práctica los conceptos fundamentales de Programación 1. La implementación de un sistema real, con datos persistentes en archivo y múltiples funcionalidades, reforzó la comprensión de la modularización como principio de diseño de software.

El uso de diccionarios como representación de entidades del dominio resultó más natural y legible que el uso de listas planas con índices numéricos. Asimismo, implementar Bubble Sort manualmente permitió comprender en profundidad la lógica de los algoritmos de ordenamiento antes de recurrir a funciones predefinidas.

Como reflexión final, el desarrollo incremental —construir, probar y validar cada función por separado— demostró ser la estrategia más efectiva para detectar errores tempranamente y mantener el código organizado.

7. Bibliografía / Webgrafía

- [1] Python Software Foundation. (2024). *Python 3 Documentation*. <https://docs.python.org/3/>
- [2] Python Software Foundation. (2024). *Built-in Types – list, dict*. <https://docs.python.org/3/library/stdtypes.html>
- [3] Python Software Foundation. (2024). *Errors and Exceptions*. <https://docs.python.org/3/tutorial/errors.html>
- [4] Python Software Foundation. (2024). *Reading and Writing Files*. <https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files>
- [5] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [6] Real Python. (2024). *Python's CSV File Reading and Writing*. <https://realpython.com/python-csv/>
- [7] Real Python. (2024). *Sorting Algorithms in Python*. <https://realpython.com/sorting-algorithms-python/>